Back to the case

Published early

Not everything has to wait until the end. The team can publish chapters, editing stages or individual functions earlier – if it plans scope, interests and approvals together.

A conversation in the team

"Why wait with the front end until the whole novel is finished?" asks the developer. "We could already show index and Digitalisate. And try out a shared section together." Falk sees the advantage: Feedback comes at a time when the team can still react to it.

Lukas hesitates: "My new publication order has not yet been published." Nora nods. Also the publisher does not want to see the later volume completely anticipated. They therefore separate the questions: Which functions can already go online? Which texts and interpretations do they release when?

Note: Early partial publications need their own release plan – not the complete completion of the edition.

Developing early – close to the edition

We start early and iteratively, but do the expansion according to the available material. A technical framework can be prepared from the start of the project. Technical views grow with a representative TEI pilot collection. Thus, the front end remains close to the respective edition stand instead of meeting a supposedly final inventory at the end.

In our case, the front end expansion follows with slight overlap on the actual edition work. This leaves little time for professional feedback before the sponsor. Falk therefore agrees at the beginning of the project an advanced technical preliminary and fixed development window with the chair's own IT. The existing capacity is re-planned; additional financial burdens are avoided. Tasks and time requirements are recorded in the updated work plan.

Guideline: Start: as early as possible, as late as necessary. Decrease core functions in time; then optimize as long as agreed capacity is available.

1 · Technical progress from project start

The technology prepares basic structure, data access, where appropriate the intended database connection and IIIF access. Test data checks interfaces and processes. Scientific cooperation is limited to the necessary basic decisions; developed test data do not determine edition hypotheses.

2 · Technical expansion with representative TEI

The pilot contains typical and difficult cases: edge addition, deletion, unclear assignment and stable image anchors. This limited model range is sufficiently described before the corresponding views are expanded. The entire TEI model does not have to be final yet. Modified rules get a versioned interface level and suitable tests.

3 · Short expansion and feedback windows

Technical editors and technicians check manageable views at the current edition stand. Feedback is bundled; not every everyday correction becomes a front-end discussion. The technology works in agreed windows instead of being bound continuously over the entire duration. Public approvals also follow the agreements with publishers and researchers.

4 · Core acceptance before end of funding, optimization afterwards within the agreed framework

The necessary core functions are removed with time for corrections. Further optimizations run as long as personnel and technical capacity are available. At an earlier necessary development end there is a viable, described version. Continuing development after job termination receives its own responsibility and capacity commitment.

Consciously limit effort

TEI remains in motion: An immediate full expansion can create unnecessary modifications. Therefore, scientific views start with a viable model range and a real pilot stock; schema, data and code are tested together.

Specialist work needs rest: Permanent surface discussions draw energy from transcription and modeling. The team bundles requirements, names a contact person and limits feedback to agreed dates.

Development binds time: Also existing IT capacity is to be planned. Technical advance, expansion and subsequent corrections are given fixed windows. The goal is long improvement, not a permanent full-time commitment of the development team.

Thus Git supports both development routes

We set up the same test basis. In the iterative way, it is often passed; in the phase model, it ensures the handovers and subsequent integration. Technical testing and technical rereading belong together.

Iterative: small changes, often examine together

1. Create a short branch for a manageable task, e.g. for the display of marginal additions. The proposed change (merge request) contains expected behavior and the affected examples.

2. With each amendment proposal, the GitLab pipeline automatically checks: Do TEI and schema fit together? Are identifiers and image references retained? Can the frontend be built? Does the interface test show the edge addition in the right place? The technology generates a access-protected test view for the editors.

3. After successful examination and technical feedback, the change is taken over to the main level. In case of a model change, we also check the corresponding adjustment of the front end. The integration protocol determines which data, schema and code commits have been tested together.

Example: Nora reports: The edge addition is missing in the interface. The amendment proposal complements view and test case. After that, Nora checks exactly this point in the test view; the pipeline also checks the previous example cases.

In phases: making handovers clear and repeatable

1. Already during the edition phase, editorial and technology arrange the transfer format. schema, representative TEI examples and expected editions are available versioned. Automatic checks report deviations, even if the actual frontend expansion starts later.

2. At the end of the phase we mark the taken delivery stand with an annotated tag, e.g. tei-uebergabe-01. The transfer log also mentions the exact commit, the schema and known open points. The development expressly refers to this version, not continuously the latest working version.

3. The first integration run builds the front end against this basis and checks the agreed example cases. Corrections run back into the relevant data as short suggestions for changes. A new transfer position replaces the old one only after a new joint examination; we do not postpone the old day.

Example: The editors provide TEI-version 01. While the technique works with it, an image reference is corrected. The correction gets its own amendment proposal and then a new transfer state. Thus, it can be seen which version the development actually uses.

Hooks: check early at work

A pre-commit hook checks the scheduled XML files before saving a commit and reports invalid markup or double identifiers. The technique holds hook and test scripts in the repo, sets them up during onboarding and documents the required tools. Local hooks must be activated on each workstation and can be bypassed. Therefore, the central pipeline repeats the mandatory tests.

Pipeline: Secure the common version

The pipeline is a test run set up on the GitLab server, not a built-in XML tester from Git. We version their configuration and use defined tool versions. Automatic tests also check the combination of the intended data, schema and code sets from the separate repos. The main branch is protected; takeovers require successful statutory checks and agreed review. Test reports and test view remain in the authorized team. A green test does not replace editorial acceptance.

Mark, repeat, selectively withdraw

We record the exact data, schema and code-commits as well as construction instructions and tool versions for each accepted integration level. Tags make such milestones easy to find. Test results and test views are backed up accordingly; short-lived pipeline artifacts are not a permanent archive. If a change causes an error, git revert can take them back by means of a new, comprehensible commit. Subsequently, the combination is checked again. Separate repos are expressly coordinated.

The first specific mission: At first, the technology sets up a versioned test call for local hooks and pipelines. The editors provide typical and difficult examples together with an expected presentation. Together, they test a valid and deliberately faulty test stand as well as the new structure of a designated integration level.

Done if: A proposal for an amendment with faulty test material is reliably stopped; a permissible version can be rebuilt and tested on the basis of its data and code identifiers.

Technical basis: Git: Hooks · GitLab: CI/CD pipelines

Three ways to combine

Functions first

index, digitalisats view or full text search can be used before the edited texts are completely available.

Example: A released index opens up manuscript pages. The search initially works on a designated transcription stock and leads to the digitalisat.

Git strategy: Frontend code, search index, index and data base receive separate version references. The public combination is released as a tested integration status.

Before the approval: Also index, search index, hit texts and picture entries belong in the approval. Not yet published interpretation is not visible unchecked.

Publish chapter by chapter

An audited subset will be published as a designated sub-issue and further chapters will be added later.

Example: A first chapter with its draft versions appears, while other volumes are still being edited.

Git strategy: Each release lists the contained units and their model/test stands. The output remains citable, even if further chapters follow.

Before the approval: Text scope, variants, comments and timing are agreed with publishers and researchers. Missing chapters are identified in the table of contents.

Show editing steps

First, a tested basic transcription will be published; genetic distinction and comments will follow in new versions.

Example: Version 0.1 provides a verified transcription. Version 0.2 supplements a first designated annotation layer; later follows the fully responsible genetic representation.

Git strategy: Transcription, annotation and editorial approval are documented per unit. New versions do not replace the cited older edition tacitly.

Before the approval: A transcription is called a transcription, not a finished genetic edition. Additional annotations get their own version and test status.

Publishing and Dissertation

The publishing house wants to see the printed book completely anticipated not long before its publication. Lukas and Nora want to determine the timing and form of their own research publications. Therefore, the team determines for each partial edition which texts, index, search hits, comments and interpretations become public. Internal testing can start earlier: Tests and feedback are also possible with a limited access. The agreed public parts then go online purposefully.

Alternative scenario: cumulative dissertations

In an alternative scenario, Lukas and Nora agree on a cumulative doctorate: Associated scientific contributions arise and appear during the research work. An early, clearly responsible publication can then lie in their own interest. Lukas could justify his new reconstruction in an independent contribution; Nora could combine her figure analysis with a published annotation version. Both can refer to the appropriate edition of the web edition. Which forms of publication, review and own parts count, is clarified early with supervision and responsible faculty according to the current doctoral regulations. This may reduce the print to have to hold everything back until a single final publication.

Cite and allocate contributions

For a concrete analysis, an archived edition with version DOI is quoted; job identifiers refer to the examined passage. On the website there are authorship and responsibility shares for the respective contribution. The further developed edition links the older version and the

contribution. A DOI makes the version referenceable; the recognition as a doctoral achievement depends on the agreed technical and formal requirements.

That's how we would start

We develop the frontend early and iterative. Basic frameworks and interface tests can be developed at the beginning of the project; the technical development starts with representative TEI data. In agreed feedback windows, we keep the digital views close to the respective edition level. Core functions are taken off in good time before the end of the project and optimize further as long as capacity is available. A released public function can be used early; chapters and editing steps follow according to the agreed approvals. For operation and maintenance a contact is named. The sentence lies with the publisher; own early sentence previews are not part of the project process in this model.

1. agree technical progress and development windows in the work plan; define a representative TEI pilot stock and a limited model range.

2. Test views at the current edition state; bundle feedback and coordinate public approvals with publisher and doctoral researchers.

3. Decrease core functions with test buffer; document further partial releases, archive references, optimizations and available connection capacity.

Done if: A first agreed offer is publicly usable, its scope and data level are recognisable, feedback has a responsible recipient and the next issue has a specific task.

Background and sources

- Cumulative Dissertation · Faculty of Humanities Göttingen: An example also for modern German literature and digital humanities. The requirements distinguish publication ability and individual contributions; the relevant doctoral regulations are decisive.

- Zenodo · DOI versioning: Versions DOIs reference a certain output; an overarching DOI can represent all versions.